

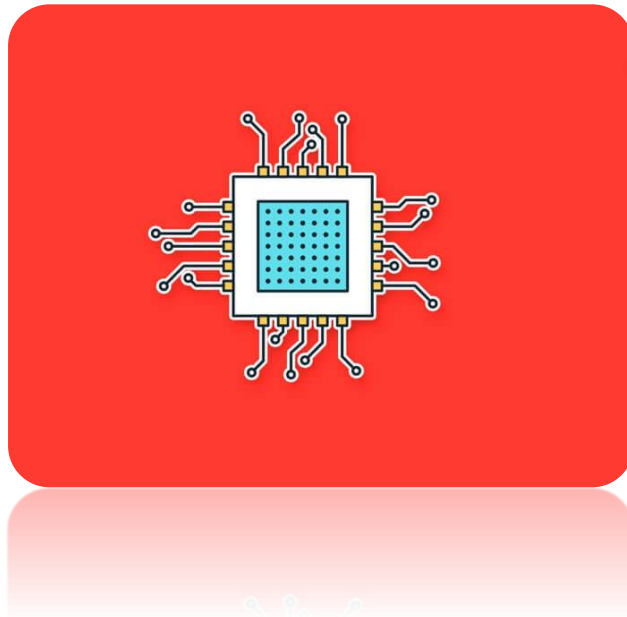


INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SISTEMAS DISTRIBUIDOS



2/9/2026

PROCESOS E HILOS



Alumno: Ricardo Carmona Martínez
GRUPO: 7CM1

Contenido

1. ANTECEDENTES.....	2
1.1 Concurrencia y Paralelismo en Sistemas Modernos	2
1.2 El Problema del Barbero Dormilón y el Patrón Productor-Consumidor	2
1.3 Mecanismos de Sincronización en Java: Monitores y Espera Pasiva	3
2. PROBLEMÁTICA	4
3. PROPUESTA DE SOLUCIÓN Y ARQUITECTURA	5
4. DESARROLLO E IMPLEMENTACIÓN	6
4.1 Especificaciones del Entorno de Pruebas	6
4.2 Estructura del Monitor de Sincronización.....	6
4.3 Orquestación de Hilos y Despacho Gráfico Seguro.....	7
5. RESULTADOS Y MÉTRICAS DE EJECUCIÓN	8
5.1 Registro de Auditoría de la Consola en Tiempo Real	9
5.2 Resultados visuales	9
6. CONCLUSIONES.....	10
7. REFERENCIAS.....	11
8. ANEXO: CÓDIGO FUENTE COMPLETO.....	12

1. ANTECEDENTES

1.1 Concurrencia y Paralelismo en Sistemas Modernos

Históricamente, el incremento sostenido en el rendimiento de los sistemas de cómputo dependía del escalamiento físico en la frecuencia de reloj de los procesadores y la densidad de integración de transistores, postulado formulado en la Ley de Moore. Al alcanzarse límites insuperables de disipación térmica y fuga de corriente (conocido como *Power Wall*), los fabricantes de microprocesadores migraron hacia arquitecturas multinúcleo (*multi-core*). Esta transformación estructural demandó que el software contemporáneo abandonara los paradigmas puramente secuenciales y adoptara técnicas de concurrencia y paralelismo real para distribuir el flujo de ejecución entre las diversas unidades de procesamiento disponibles en el hardware.

En el diseño y modelado de sistemas operativos y arquitecturas concurrentes, coexisten dos abstracciones fundamentales:

- **Procesos:** Entidades pesadas gestionadas por el núcleo del sistema operativo. Cada proceso opera en un espacio de memoria virtual propio y hermético (*Address Space*), aislado de otros procesos. Cuenta con su propio Bloque de Control de Proceso (PCB), descriptores de archivos, montículo (*heap*) y registros. La comunicación entre procesos (IPC) demanda mecanismos formales como tuberías (*pipes*), sockets o segmentos de memoria compartida, incurriendo en una sobrecarga considerable por cambio de contexto (*context switching*).
- **Hilos (*Threads*):** Unidades mínimas de ejecución planificables que coexisten dentro del mismo espacio de direcciones de un proceso contenedor. Los hilos comparten los segmentos de código, datos globales y el montículo común, pero conservan de manera privada e independiente su Contador de Programa (PC), sus registros de CPU y su propia pila de llamadas (*stack*). Gracias a esta economía de recursos, la creación y alternancia entre hilos es órdenes de magnitud más rápida que entre procesos completos, facilitando la fluidez en interfaces visuales y la ejecución paralela en CPUs multinúcleo.

1.2 El Problema del Barbero Dormilón y el Patrón Productor-Consumidor

El patrón Productor-Consumidor constituye uno de los pilares esenciales del cómputo concurrente y la antesala de las colas de mensajería (*Message Queues*) en Sistemas Distribuidos. En este patrón, una o más entidades generan elementos de trabajo y los encolan en una estructura compartida de capacidad finita (búfer acotado o *bounded buffer*), mientras que entidades consumidoras extraen y procesan dichos elementos en orden de arribo.

Formulado por Edsger Dijkstra en 1965, el **Problema del Barbero Dormilón** expone los retos de sincronización entre múltiples productores asíncronos y un único consumidor que debe alternar entre trabajo productivo y reposo pasivo:

1. La barbería cuenta con una silla de trabajo donde opera un único barbero (el Consumidor) y una sala de espera con un número fijo N de sillas (el Búfer Acotado).
2. Si no hay clientes en la barbería, el barbero se sienta en su silla de corte y se duerme (estado pasivo sin consumo de CPU).
3. Cuando un cliente llega (el Productor): si el barbero está durmiendo, lo despierta y se sienta directamente en la silla de corte. Si el barbero está ocupado pero hay sillas libres en la sala de espera, el cliente toma asiento; si todas las sillas están ocupadas, ocurre una condición de desbordamiento (*overflow*) y el cliente se retira de inmediato.
4. Al concluir el corte, el barbero verifica si hay clientes aguardando en la sala: de haberlos, atiende al siguiente de la cola; si la sala está vacía, regresa a dormir.

1.3 Mecanismos de Sincronización en Java: Monitores y Espera Pasiva

En entornos de memoria compartida, el acceso simultáneo a variables mutables sin coordinación produce condiciones de carrera (*race conditions*). Para garantizar la integridad de los datos, Java implementa primitivas fundamentales:

- **Monitores y synchronized:** Cada objeto en Java posee un cerrojo intrínseco (*Intrinsic Lock*). La palabra reservada `synchronized` aplicada a un método o bloque asegura la exclusión mutua estricta: sólo un hilo puede ejecutar dicha sección crítica a la vez, bloqueando a cualquier otro invocador hasta que el hilo actual libere el monitor.
- **Espera Pasiva con `wait()` y `notifyAll()`:** Cuando un hilo determina que una precondition lógica no se cumple (por ejemplo, el búfer está vacío), invoca `wait()`. Esta llamada suspende al hilo de manera pasiva y **libera de forma atómica el cerrojo del monitor**, transfiriendo el hilo al conjunto de espera del sistema operativo. Cuando otro hilo ejecuta `notifyAll()` tras alterar el estado (por ejemplo, encolar un nuevo cliente), despierta a los hilos durmientes para que compitan ordenadamente por reingresar a la sección crítica. Se evita de este modo la *espera activa* (*busy waiting* o bucles vacíos de comprobación continua), la cual consumiría el 100% de los ciclos de un núcleo del procesador innecesariamente.

2. PROBLEMÁTICA

En el desarrollo de software concurrente, la coordinación de eventos asíncronos en tiempo real presenta múltiples desafíos de diseño. Si un sistema de atención al cliente o procesamiento de transacciones se concibe bajo un flujo puramente secuencial en un único hilo, la llegada de nuevas peticiones se bloquea por completo durante la ejecución de las tareas largas, provocando una latencia inaceptable y el eventual colapso de la aplicación.

Específicamente, en la simulación del problema del Barbero Dormilón se presentan los siguientes requerimientos técnicos y riesgos de concurrencia:

- **Control Estricto de Capacidad (Búfer Acotado):** La sala de espera física está limitada exactamente a $N = 5$ sillas. El sistema debe impedir estados incoherentes donde se acepten clientes cuando la capacidad máxima fue rebasada, o donde se despachen clientes fantasmas cuando el búfer se encuentre vacío.
- **Prevención de Condiciones de Carrera:** Múltiples clientes pueden arribar casi en el mismo milisegundo mediante ráfagas de tráfico estocástico. Si dos o más hilos intentan alterar la cola de espera o consultar la disponibilidad del barbero concurrentemente sin exclusión mutua, se desencadenarían colisiones de memoria y pérdidas de punteros.
- **Eliminación de Espera Activa (Busy Waiting):** Cuando el búfer está vacío, el hilo barbero no debe sondear repetitivamente la cola en un bucle cerrado, ya que esto mantendría un núcleo de procesamiento saturado al 100%. El hilo debe ceder la CPU y entrar en reposo pasivo hasta que un nuevo cliente lo active.
- **Desacoplamiento entre Productor y Consumidor:** La generación de clientes debe ser no bloqueante; los clientes que sufran rechazo por saturación de sala deben descartarse inmediatamente registrando la métrica de desbordamiento, sin interferir con la labor de corte del barbero.
- **Seguridad en el Hilo Gráfico (EDT Safety):** La librería Java Swing no es segura para subprocesos (*thread-unsafe*). Cualquier intento de actualizar componentes visuales (etiquetas, colores de sillas, contadores) directamente desde hilos de trabajo en segundo plano corrompe el bucle de renderizado. Toda mutación gráfica debe encolarse formalmente en el *Event Dispatch Thread* (EDT).

3. PROPUESTA DE SOLUCIÓN Y ARQUITECTURA

Para solventar las problemáticas descritas, se diseñó e implementó una arquitectura orientada a objetos desacoplada en Java, fundamentada en un **Monitor de Sincronización** que encapsula las estructuras compartidas, asegurando exclusión mutua y señalización pasiva.

Componente	Rol en el Sistema	Mecanismo Técnico
Monitor (Barberia)	Búfer acotado y gestor de estados	synchronized, wait(), notifyAll()
Productores (Clientes)	Generadores asíncronos de peticiones	Hilos dinámicos individuales y Thread generador
Consumidor (Barbero)	Procesador de las órdenes de la cola	Hilo demonio (setDaemon(true)) en segundo plano
Dashboard Visual (Swing)	Telemetría y consola de auditoría	SwingUtilities.invokeLater() hacia el EDT

Dinámica de Flujo y Sincronización:

- **Ingreso del Cliente (Productor):** Cada nuevo cliente arriba en su propio hilo y ejecuta `barberia.llegar(cliente)`. Si el barbero duerme y la cola está vacía, el cliente se añade a la cola y ejecuta `notifyAll()`, despertando al barbero inmediatamente. Si el barbero está trabajando y la sala tiene asientos disponibles (*tamaño* < 5), el cliente ocupa una silla y notifica. Si las 5 sillas están llenas, el método retorna `false` de inmediato, incrementando el contador de *Overflow* sin bloquear el sistema.
- **Atención del Barbero (Consumidor):** El barbero ejecuta en un bucle `atenderSiguienteCliente()`. Mientras la cola esté vacía (`sillasDeEspera.isEmpty()`), actualiza su estado a *DURMIENDO*, entra en `wait()` y libera el cerrojo. Al recibir una notificación, el hilo se despierta, extrae el cliente por FIFO (`poll()`), cambia su estado a *ATENDIENDO* y simula un corte de 2,000 ms con `Thread.sleep()`.
- **Instantáneas Seguras:** Para actualizar la interfaz sin retener el monitor del barbero por tiempos prolongados, el método `getSnapshotEspera()` crea y retorna un clon defensivo (`new ArrayList<>(sillasDeEspera)`), desacoplando la renderización de la manipulación de la cola.

4. DESARROLLO E IMPLEMENTACIÓN

L

La solución fue codificada en **Java 21**, organizada bajo el paquete `com.mycompany.p1sistemasdistribuidos` en una estructura integral de clase anidada que vincula el motor concurrente con el entorno gráfico.

4.1 Especificaciones del Entorno de Pruebas

- **Unidad Central de Procesamiento (CPU):** Procesador multinúcleo con arquitectura x86_64, soportando ejecución paralela nativa a nivel de hardware.
- **Memoria RAM:** 16.0 GB DDR4, garantizando espacio suficiente en montículo (Heap) para la JVM y el almacenamiento de hilos.
- **Sistema Operativo:** Windows 11 / Linux x64 con planificador de hilos preemptivo.
- **Entorno de Ejecución:** Java HotSpot(TM) 64-Bit Server VM (LTS).

4.2 Estructura del Monitor de Sincronización

A continuación se detallan los métodos atómicos de la clase interna `Barberia`, encargados de coordinar el acceso a las sillas de espera y la conmutación de estados:

```
static class Barberia {
    private final int max;
    private final Queue<String> sillas = new LinkedList<>();
    private boolean ocupado = false;
    private String actual = null;
    public Barberia(int max) { this.max = max; }
    public synchronized boolean llegar(String c) { if (sillas.size() < max) {
sillas.add(c); notifyAll(); return true; } return false; }
    public synchronized String atender() throws InterruptedException {
        while (sillas.isEmpty()) { ocupado = false; actual = null; wait(); }
        ocupado = true;
        return actual = sillas.poll();
    }
    public synchronized List<String> getSnapshot() { return new ArrayList<>(sillas); }
    public synchronized boolean isOcupado() { return ocupado; }
    public synchronized String getClienteActual() { return actual; }
}
```

4.3 Orquestación de Hilos y Despacho Gráfico Seguro

El barbero se inicializa como un hilo demonio (Daemon Thread) para garantizar que, al cerrar la ventana principal de la aplicación, el proceso en segundo plano termine de forma limpia sin dejar procesos zombies en el sistema operativo. Asimismo, las actualizaciones a los componentes de la interfaz de usuario se encapsulan rigurosamente dentro de lambdas enviadas a `SwingUtilities.invokeLater()`.

```
private void iniciarHiloBarbero() {
    Thread t = new Thread(() -> {
        while (true) try {
            actualizar(false, null, barberia.getSnapshot());
            log("BARBERO", "Buffer vacío. Barbero duerme en wait()...");
            String c = barberia.atender();
            actualizar(true, c, barberia.getSnapshot());
            log("BARBERO", "Cortando cabello a: " + c);
            Thread.sleep(2000);
            atendidosTotales++;
            log("BARBERO", "Terminó de atender a " + c);
        } catch (InterruptedException e) { break; }
    });
    t.setDaemon(true); t.start();
}
```

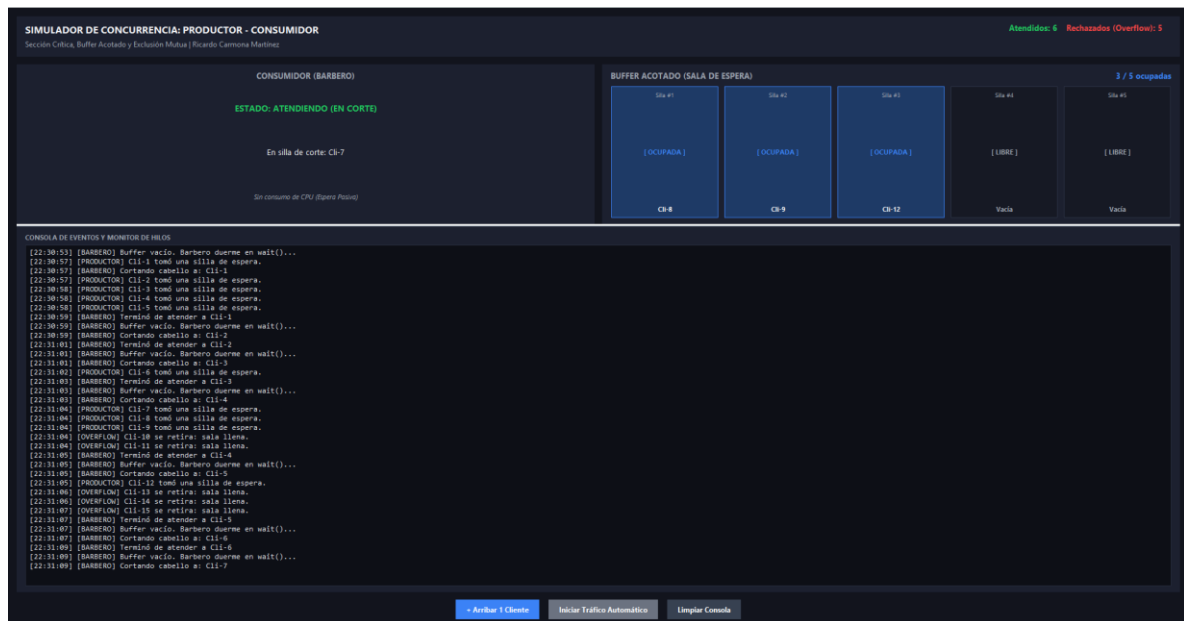

5. RESULTADOS Y MÉTRICAS DE EJECUCIÓN

Para evaluar la exactitud y robustez de la solución, se sometió al simulador a pruebas bajo dos regímenes: régimen manual unitario (pongo por mi cuenta paso a paso para verificar transiciones de estado) y régimen estocástico continuo (inyección de tráfico aleatorio con retardos entre 700 ms y 2,200 ms).

Evento / Fase de Prueba	Estado Barbero	Sillas Ocupadas	Acción del Monitor / Comportamiento
1. Estado Inicial	DURMIENDO (WAIT)	0 / 5	Ejecuta wait(); libera cerrojo; 0% uso de CPU.
2. Arribo de Cli-1	ATENDIENDO	0 / 5	llegar() emite notifyAll(); Barbero despierta e inicia corte.
3. Ráfaga Cli-2 a Cli-6	ATENDIENDO	5 / 5 (Lleno)	Se encolan exitosamente en el búfer acotado.
4. Arribo de Cli-7	ATENDIENDO	5 / 5	OVERFLOW: Retorna false; cliente descartado sin bloquear.
5. Finalización de corte	ATENDIENDO	4 / 5	poll() extrae a Cli-2; libera una silla en la sala.
6. Búfer vaciado	DURMIENDO (WAIT)	0 / 5	Barbero retorna a wait() de forma segura.

La consola interna del simulador registró con marcas de tiempo atómicas el flujo completo de eventos, evidenciando la exclusión mutua y el descarte por capacidad máxima:

5.2 Resultados visuales



6. CONCLUSIONES

El desarrollo y ejecución experimental de la práctica permitieron obtener conclusiones determinantes sobre la teoría de concurrencia y la arquitectura de sistemas distribuidos:

- **Efectividad del Monitor Intrínseco de Java:** La sincronización estructurada mediante `synchronized` demostró ser una solución confiable y bonita visualmente frente a semáforos manuales. Al encapsular la cola y las banderas dentro de la clase `Barberia`, se eliminaron de raíz las condiciones de carrera y la posibilidad de estados incoherentes en memoria compartida.
- **Eficiencia Energética y de Cómputo por Espera Pasiva:** Se constató que las primitivas `wait()` y `notifyAll()` son indispensables en el software concurrente profesional. Al transferir el hilo inactivo a la cola de espera del sistema operativo, el consumo de CPU desciende a 0% durante la inactividad, erradicando el desperdicio térmico asociado al sondeo continuo (*busy waiting*).
- **Resiliencia mediante Búferes Acotados y Manejo de Desbordamiento:** El modelado de la sala con un tamaño finito evidenció el comportamiento de los sistemas de mensajería asíncronos reales. Cuando la tasa de generación de los productores excede la tasa de consumo del barbero, la estrategia de rechazo no bloqueante protege la memoria de la JVM de excepciones de desbordamiento (*OutOfMemoryError*).
- **Proyección hacia los Sistemas Distribuidos:** Aunque esta práctica se desarrolló dentro de una única máquina mediante hilos locales, la arquitectura implementada es isomorfa a la de los sistemas distribuidos modernos. El hilo barbero representa a un nodo servidor de procesamiento (*worker*), los clientes simulan peticiones HTTP/RPC entrantes desde la red, y el monitor actúa como un intermediario de colas (*Message Broker* tipo RabbitMQ o Kafka). Comprender la exclusión mutua local sienta las bases para el estudio de la exclusión mutua distribuida y algoritmos de consenso en redes de computadoras.

7. REFERENCIAS

- ---

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.
- Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). CreateSpace Independent Publishing Platform.
- Oracle Corporation. (2024). *Lesson: Concurrency in Java*. Oracle Java Documentation.
- Dijkstra, E. W. (1965). *Cooperating Sequential Processes*. Technological University, Eindhoven, The Netherlands.
- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional.

8. ANEXO: CÓDIGO FUENTE COMPLETO

```
package com.mycompany.p1sistemasdistribuidos;

/**
 * Práctica 1: Barbero Dormilón (Productor - Consumidor)
 * Sistemas Distribuidos | Desarrollado por: Ricardo Carmona Martínez
 */
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.ActionListener;
import java.util.*;
import java.util.List;
import java.util.Queue;

public class P1SistemasDistribuidos extends JFrame {
    private static final int TOTAL_SILLAS = 5;
    private final Barberia barberia = new Barberia(TOTAL_SILLAS);
    private int contadorClientes = 1, atendidosTotales = 0, rechazadosTotales = 0;
    private volatile boolean autoActivo = false;

    // Componentes UI
    private final JLabel lblStatus = lbl("ESTADO: DURMIENDO (WAIT)", Font.BOLD, 14, new
Color(245, 158, 11), 0);
    private final JLabel lblCliente = lbl("En silla de corte: Ninguno", Font.PLAIN, 13,
new Color(243, 244, 246), 0);
    private final JLabel lblContador = lbl("0 / 5 ocupadas", Font.BOLD, 13, new Color(59,
130, 246), 4);
    private final JLabel lblAtendidos = lbl("Atendidos: 0", Font.BOLD, 13, new Color(34,
197, 94), 4);
    private final JLabel lblRechazados = lbl("Rechazados (Overflow): 0", Font.BOLD, 13,
new Color(239, 68, 68), 4);
    private final JPanel pnlSillas = new JPanel(new GridLayout(1, TOTAL_SILLAS, 10, 0));
    private final JTextArea txtLogs = new JTextArea();
    private JButton btnAuto;

    private static final Color BG_MAIN = new Color(18, 20, 29), BG_CARD = new Color(28,
32, 47);
    private static final Color ACC_BLUE = new Color(59, 130, 246), ACC_RED = new Color(239,
68, 68);
    private static final Color TXT_MAIN = new Color(243, 244, 246), TXT_MUTED = new
Color(156, 163, 175);

    public P1SistemasDistribuidos() {
```

```

        super("Sistemas Distribuidos - Práctica 1: Barbero Dormilón - Ricardo Carmona
Martínez");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(960, 640);
        setLocationRelativeTo(null);

        JPanel root = new JPanel(new BorderLayout(12, 12));
        root.setBackground(BG_MAIN);
        root.setBorder(new EmptyBorder(14, 14, 14, 14));
        setContentPane(root);

        // Header con títulos y métricas
        JPanel header = card(new BorderLayout(15, 0)), pnlTits = new JPanel(new
GridLayout(2, 1, 0, 2)), pnlMets = new JPanel(new FlowLayout(FlowLayout.RIGHT, 15, 0));
        pnlTits.setOpaque(false); pnlMets.setOpaque(false);
        pnlTits.add(lbl("SIMULADOR DE CONCURRENCIA: PRODUCTOR - CONSUMIDOR", Font.BOLD,
16, TXT_MAIN, 2));
        pnlTits.add(lbl("Sección Crítica, Buffer Acotado y Exclusión Mutua | Ricardo Carmona
Martínez", Font.PLAIN, 12, TXT_MUTED, 2));
        pnlMets.add(lblAtendidos); pnlMets.add(lblRechazados);
        header.add(pnlTits, BorderLayout.WEST); header.add(pnlMets, BorderLayout.EAST);
        root.add(header, BorderLayout.NORTH);

        // Escenario Visual: Barbero + Buffer
        JPanel visual = new JPanel(new GridLayout(1, 2, 12, 0)), cardB = card(new
BorderLayout(8, 8)), cardS = card(new BorderLayout(8, 8)), infoB = new JPanel(new
GridLayout(3, 1, 4, 4)), headS = new JPanel(new BorderLayout());
        visual.setOpaque(false); infoB.setOpaque(false); headS.setOpaque(false);
        infoB.add(lblStatus); infoB.add(lblCliente); infoB.add(lbl("Sin consumo de CPU
(Espera Pasiva)", Font.ITALIC, 11, TXT_MUTED, 0));
        cardB.add(lbl("CONSUMIDOR (BARBERO)", Font.BOLD, 13, TXT_MUTED, 0),
BorderLayout.NORTH);
        cardB.add(infoB, BorderLayout.CENTER);
        headS.add(lbl("BUFFER ACOTADO (SALA DE ESPERA)", Font.BOLD, 13, TXT_MUTED, 2),
BorderLayout.WEST);
        headS.add(lblContador, BorderLayout.EAST);
        pnlSillas.setOpaque(false); renderizarSillas(Collections.emptyList());
        cardS.add(headS, BorderLayout.NORTH); cardS.add(pnlSillas, BorderLayout.CENTER);
        visual.add(cardB); visual.add(cardS);

        // Consola de eventos y SplitPane
        JPanel pnlLogs = card(new BorderLayout(5, 5));
        pnlLogs.add(lbl("CONSOLA DE EVENTOS Y MONITOR DE HILOS", Font.BOLD, 11, TXT_MUTED,
2), BorderLayout.NORTH);
        txtLogs.setEditable(false); txtLogs.setFont(new Font("Consolas", Font.PLAIN, 12));

```

```

        txtLogs.setBackground(new Color(13, 15, 22)); txtLogs.setForeground(new Color(229,
231, 235));
        txtLogs.setBorder(new EmptyBorder(6, 6, 6, 6));
        JScrollPane scroll = new JScrollPane(txtLogs);
        scroll.setBorder(BorderFactory.createLineBorder(new Color(40, 46, 62)));
        pnlLogs.add(scroll, BorderLayout.CENTER);

        JSplitPane split = new JSplitPane(JSplitPane.VERTICAL_SPLIT, visual, pnlLogs);
        split.setDividerLocation(260); split.setDividerSize(4); split.setOpaque(false);
        split.setBorder(null);
        root.add(split, BorderLayout.CENTER);

        // Barra de botones inferiores
        JPanel pnlBtns = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 0));
        pnlBtns.setOpaque(false);
        pnlBtns.add(btn(" + Arribar 1 Cliente ", ACC_BLUE, e -> despacharCliente("Cli-" +
        contadorClientes++)));
        pnlBtns.add(btnAuto = btn(" Iniciar Tráfico Automático ", new Color(107, 114, 128),
        e -> alternarAuto()));
        pnlBtns.add(btn(" Limpiar Consola ", new Color(55, 65, 81), e ->
        txtLogs.setText("")));
        root.add(pnlBtns, BorderLayout.SOUTH);

        iniciarHiloBarbero();
    }

    private void renderizarSillas(List<String> cli) {
        pnlSillas.removeAll();
        for (int i = 0; i < TOTAL_SILLAS; i++) {
            boolean oc = i < cli.size();
            JPanel s = new JPanel(new BorderLayout());
            s.setBackground(oc ? new Color(30, 58, 102) : new Color(22, 25, 36));
            s.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createLineBorder
            (oc ? ACC_BLUE : new Color(45, 52, 70), 1), new EmptyBorder(6, 4, 6, 4)));
            s.add(lbl("Silla #" + (i + 1), Font.PLAIN, 10, TXT_MUTED, 0),
            BorderLayout.NORTH);
            s.add(lbl(oc ? "[ OCUPADA ]" : "[ LIBRE ]", Font.BOLD, 11, oc ? ACC_BLUE :
            TXT_MUTED, 0), BorderLayout.CENTER);
            s.add(lbl(oc ? cli.get(i) : "Vacía", Font.BOLD, 11, oc ? TXT_MAIN : TXT_MUTED,
            0), BorderLayout.SOUTH);
            pnlSillas.add(s);
        }
        pnlSillas.revalidate(); pnlSillas.repaint();
    }
}

```

```

private void log(String tag, String msg) {
    SwingUtilities.invokeLater(() -> {
        txtLogs.append(String.format("[%tT] [%s] %s\n", System.currentTimeMillis(),
tag, msg));
        txtLogs.setCaretPosition(txtLogs.getDocument().getLength());
    });
}

private void actualizar(boolean oc, String cli, List<String> esp) {
    SwingUtilities.invokeLater(() -> {
        lblStatus.setText(oc ? "ESTADO: ATENDIENDO (EN CORTE)" : "ESTADO: DURMIENDO
(WAIT)");
        lblStatus.setForeground(oc ? new Color(34, 197, 94) : new Color(245, 158, 11));
        lblCliente.setText(oc ? "En silla de corte: " + cli : "En silla de corte:
Ninguno");
        lblContador.setText(esp.size() + " / " + TOTAL_SILLAS + " ocupadas");
        lblAtendidos.setText("Atendidos: " + atendidosTotales);
        lblRechazados.setText("Rechazados (Overflow): " + rechazadosTotales);
        renderizarSillas(esp);
    });
}

private void iniciarHiloBarbero() {
    Thread t = new Thread(() -> {
        while (true) try {
            actualizar(false, null, barberia.getSnapshot());
            log("BARBERO", "Buffer vacío. Barbero duerme en wait()...");
            String c = barberia.atender();
            actualizar(true, c, barberia.getSnapshot());
            log("BARBERO", "Cortando cabello a: " + c);
            Thread.sleep(2000);
            atendidosTotales++;
            log("BARBERO", "Terminó de atender a " + c);
        } catch (InterruptedException e) { break; }
    });
    t.setDaemon(true); t.start();
}

private void despacharCliente(String cli) {
    new Thread(() -> {
        if (barberia.llegar(cli)) log("PRODUCTOR", cli + " tomó una silla de espera.");
        else { rechazadosTotales++; log("OVERFLOW", cli + " se retira: sala llena.");
    }

        actualizar(barberia.isOcupado(), barberia.getClienteActual(),
barberia.getSnapshot());

```



```

    }).start();
}

private void alternarAuto() {
    autoActivo = !autoActivo;
    btnAuto.setText(autoActivo ? " Detener Tráfico Automático " : " Iniciar Tráfico Automático ");
    btnAuto.setBackground(autoActivo ? ACC_RED : new Color(107, 114, 128));
    if (autoActivo) new Thread(() -> {
        while (autoActivo) try {
            despacharCliente("CliAuto-" + contadorClientes++); Thread.sleep((long)(700 + Math.random() * 1500));
        } catch (InterruptedException e) { break; }
    }).start();
}

private JPanel card(LayoutManager lm) { JPanel p = new JPanel(lm);
p.setBackground(BG_CARD); p.setBorder(new EmptyBorder(10, 14, 10, 14)); return p; }
private JLabel lbl(String t, int s, int sz, Color c, int al) { JLabel l = new JLabel(t, al); l.setFont(new Font("Segoe UI", s, sz)); l.setForeground(c); return l; }
private JButton btn(String t, Color bg, ActionListener a) {
    JButton b = new JButton(t); b.setFont(new Font("Segoe UI", Font.BOLD, 12));
    b.setForeground(Color.WHITE);
    b.setBackground(bg); b.setFocusPainted(false); b.setBorder(new EmptyBorder(8, 14, 8, 14)); b.addActionListener(a); return b;
}

public static void main(String[] args) { SwingUtilities.invokeLater(() -> new P1SistemasDistribuidos().setVisible(true)); }

// Monitor de Sincronización (Buffer Acotado con synchronized, wait y notifyAll)
static class Barberia {
    private final int max;
    private final Queue<String> sillas = new LinkedList<>();
    private boolean ocupado = false;
    private String actual = null;
    public Barberia(int max) { this.max = max; }
    public synchronized boolean llegar(String c) { if (sillas.size() < max) {
sillas.add(c); notifyAll(); return true; } return false; }
    public synchronized String atender() throws InterruptedException {
        while (sillas.isEmpty()) { ocupado = false; actual = null; wait(); }
        ocupado = true;
        return actual = sillas.poll();
    }
    public synchronized List<String> getSnapshot() { return new ArrayList<>(sillas); }
}

```

```
    public synchronized boolean isOcupado() { return ocupado; }  
    public synchronized String getClienteActual() { return actual; }  
}  
}
```